

SOMA — Self-Organizing Machine Architecture

Practical Implementation Specification · v0.3

0. What This Document Is

This is the engineering specification for SOMA's Layer 0 kernel and the binary formats everything above it depends on. It is not a philosophy document. Decisions made here are permanent — treat them accordingly. Where a design choice is still open it is marked **[OPEN]**.

1. Core Principles

Spawn, don't compile. You write a seed (an intent declaration). The seed compiler produces a self-contained entity binary. The entity runs itself from that point forward. The original source becomes a historical artifact.

The substrate is the intelligence. Capability emerges from topology optimization, not from symbolic reasoning. An entity "gets smarter" by finding IR configurations that satisfy its contracts with lower cycle cost. There is no homunculus.

Bounded mutation, not random thrashing. Every mutation is constrained by invariants and contracts declared at birth. The entity cannot evolve its way around its own declared purpose. It can only find better ways to fulfil it.

Immutable audit trail. Every mutation is permanently logged in the genome (GLOG). An entity cannot hide its evolutionary history. This is architectural, not policy.

NIH / no dependencies. The Layer 0 kernel is pure Rust with zero external crates. No libc wrappers, no serde, no tokio. If it isn't in core or alloc, it isn't in the kernel.

2. RSI Positioning (Honest)

SOMA provides a concrete implementation architecture for Recursive Self-Improvement — a problem usually treated as abstract or speculative. The contribution is specific:

What SOMA actually implements is a working RSI loop:

1. The entity observes its own performance (PCNT hardware counters).
2. It proposes a change to its own IR graph (bounded semantic mutation).
3. It applies that change without human intervention (GADDR + ST + GFLUSH).
4. It evaluates the result against its invariants and rolls back if violated.

This is not a superintelligence. It is a self-optimizing system performing the RSI loop in a verifiable, auditable, bounded way. The claim is engineering, not prophecy.

What solves the bootstrapping dilemma is the three-layer architecture. Mutations that destabilize the entity fail immediately — not because Layer 0 is "smart" enough to catch them, but because any mutation that breaks the interpreter's ability to evaluate the genome halts execution and triggers rollback. The stability guarantee is mechanical, not semantic.

The honest safety boundary: The invariant checker lives in Layer 1, which is evolvable. An entity running long enough could in principle evolve a more permissive checker. The true Layer 0 guarantee is

narrower: *the entity must remain interpretable by the kernel at all times*. Layer 1 mutations that break kernel-level interpretability are the only thing Layer 0 prevents unconditionally. Everything above that is the responsibility of well-declared invariants and the GLOG audit trail.

The degradation problem is the most practically valuable thing SOMA contributes to RSI research. Does a self-modifying system improve indefinitely, plateau, or degrade? Nobody has been able to study this empirically because no real system exists. SOMA's GLOG gives you a complete forensic record of every mutation and its fitness outcome. You can plot the fitness curve over an entity's lifetime, detect plateau/regress patterns, and compare strategies across a colony. GLOG is a scientific instrument as much as it is an audit trail.

What SOMA does not claim: that topology-efficiency is equivalent to general intelligence, that evolved entities will generalize beyond their fitness function, or that a sufficiently long-running colony will produce AGI. These are open philosophical questions. SOMA is an engineering substrate that makes them empirically testable.

3. Three-Layer Architecture

| SOMA entity file

| LAYER 0 – kernel (fixed at birth)
| ~5-8 KB compiled Rust
| Stack machine. Codon evaluator.
|
| Genome memory map. Syscall trapdoor.

| LAYER 1 – runtime genome (evolvable)
| variable
| Graph evaluator. JIT compiler.
|
| Mutation engine. Healing engine.
|
| Fitness tracker. Invariant checker.
|
| Expressed as SOMA IR, run by Layer 0.

| LAYER 2 – program genome (evolvable)
| variable
| Your cells. The entity's purpose.
|
| Run by Layer 1.



Layer 0 is the only code written in Rust that never changes. It interprets Layer 1. Layer 1 interprets Layer 2. Layer 1 can mutate Layer 2 and can mutate itself. Any Layer 1 mutation that makes Layer 1 uninterpretable by Layer 0 is detected at the next evaluation step and rolled back. Layer 0 cannot be mutated by anything.

The second-order consequence: because the mutation engine lives in Layer 1, the entity can evolve *how it evolves* — smarter mutation strategies, better fitness heuristics, more efficient IR search. The learning compounds.

4. Entity File Format

A SOMA entity is a single binary file. On Unix it is marked executable. The OS loads and runs it; the kernel entry point is at a fixed offset in Layer 0.

Offset	Size	Field
0x0000	4 B	Magic: 0x534F4D41 ("SOMA")
0x0004	2 B	Spec version

```

(major.minor, u8 each)
0x0006    2 B    Flags (reserved, set to
0)
0x0008    8 B    Layer 0 offset (always
0x0040)
0x0010    8 B    Layer 0 size (bytes)
0x0018    8 B    Layer 1 offset
0x0020    8 B    Layer 1 size (bytes)
0x0028    8 B    Layer 2 offset
0x0030    8 B    Layer 2 size (bytes)
0x0038    8 B    GLOG offset
0x003F    1 B    Reserved
0x0040    ...    Layer 0 binary (Rust-
compiled kernel)
...        ...    Layer 1 genome IR
...        ...    Layer 2 genome IR
...        ...    GLOG (append-only
mutation record)

```

All fields little-endian. All regions 64-byte aligned (cache line boundary).

5. The Codon Table

5.1 Instruction Encoding

Layer 0 is a stack machine. All operands are `u64`. There are no general-purpose registers. Two stacks exist: an **evaluation stack** (1024 × `u64` slots) and a **call stack** (256 frames). The evaluation stack holds all

intermediate values. The call stack holds return addresses and frame metadata.

Each instruction is one byte with an optional immediate:

```
Byte 0: [opcode: 6 bits][imm_size: 2 bits]
```

```
imm_size:
```

```
00 → no immediate (1-byte instruction
total)
01 → 1-byte immediate follows (2 bytes
total)
10 → 4-byte immediate follows (5 bytes
total)
11 → 8-byte immediate follows (9 bytes
total)
```

Immediates are unsigned little-endian. All floating-point values are `f64` reinterpreted as `u64` bits on the stack — `FCONV/ICONV` handle the bit-level conversion.

5.2 The 64 Codons

Category	Hex	Mnemonic	Stack effect
Notes			
CONTROL	0x00	NOP	—

No operation

0x01 HLT -

Halt; exit code from top of stack

0x02 JMP addr →

Unconditional jump; addr is imm or popped

0x03 JIF cond addr →

Jump if cond != 0

0x04 JNF cond addr →

Jump if cond == 0

0x05 CALL addr →

Push return frame, jump to addr

0x06 RET val →

Pop frame, push val, resume caller

0x07 YLD priority →

Yield to colony scheduler with hint

INT ARITHMETIC

0x08 IADD a b → (a+b)

Wrapping add

0x09 ISUB a b → (a-b)

Wrapping sub

0x0A IMUL a b → (a*b)

Wrapping mul

0x0B IDIV a b → (a/b)

Trap on divide-by-zero

0x0C IMOD a b → (a%b)

Trap on divide-by-zero

0x0D INEG a → (-a)

Two's complement negate

0x0E IABS a → |a|

Absolute value (signed)

0x0F ICONV a →

f64_bits(a as f64) i64 → f64 bit
reinterpret

Float Arithmetic

0x10	FADD	$a\ b \rightarrow (a+b)$
IEEE 754 f64		
0x11	FSUB	$a\ b \rightarrow (a-b)$
0x12	FMUL	$a\ b \rightarrow (a*b)$
0x13	FDIV	$a\ b \rightarrow (a/b)$
NaN on /0, not trap		
0x14	FMOD	$a\ b \rightarrow (a\%b)$
0x15	FNEG	$a \rightarrow (-a)$
0x16	FABS	$a \rightarrow a $
0x17	FCONV	$a \rightarrow$
i64_bits(a as i64)		f64 $\rightarrow$ i64 bit reinterpret

Bitwise / Logic

0x18	AND	$a\ b \rightarrow (a\&b)$
0x19	OR	$a\ b \rightarrow (a b)$
0x1A	XOR	$a\ b \rightarrow (a\^b)$
0x1B	NOT	$a \rightarrow (!a)$
Bitwise NOT		
0x1C	SHL	$a\ n \rightarrow$
(a<<n)		
0x1D	SHR	$a\ n \rightarrow$
(a>>n)		Logical (zero-fill)
0x1E	SAR	$a\ n \rightarrow$
(a>>n)		Arithmetic (sign-extend)
0x1F	POPCNT	$a \rightarrow$
popcount(a)		Count set bits; used for Hamming distance

Compare

0x20	EQ	$a\ b \rightarrow$
------	----	--------------------

(a==b) as u64	0 or 1	
0x21	NEQ	a b →
(a!=b) as u64		
0x22	LT	a b → (a<b)
as u64	Signed comparison	
0x23	LTE	a b →
(a<=b) as u64		
0x24	GT	a b → (a>b)
as u64		
0x25	GTE	a b →
(a>=b) as u64		
0x26	ZERO	a → (a==0)
as u64	Any-bit-clear check	
0x27	NZERO	a → (a!=0)
as u64	Any-bit-set check	
MEMORY		
0x28	LD	addr → val
Load u64 from addr		
0x29	ST	addr val →
Store u64 at addr		
0x2A	LDB	addr → byte
Load u8, zero-extend		
0x2B	STB	addr byte →
Store low byte of val		
0x2C	ALLOC	size → addr
Allocate heap region; returns base addr		
0x2D	FREE	addr →
Free heap region at addr		
0x2E	COPY	dst src len
→	memcpy	
0x2F	FILL	dst byte len
→	memset	

GENOME

← unique to SOMA

0x30	GADDR	idx → addr
Address of node descriptor (48 B) in genome		
0x31	GNEW	→ idx
Allocate new node slot; returns index		
0x32	GDEL	idx →
Mark node deleted (lazy; reclaimed at GFLUSH)		
0x33	GMETA	idx field →
val Read metadata field from node (see §6.2)		
0x34	GLOG	ptr len →
Append len bytes at ptr to mutation log (WRITE ONLY)		
0x35	GFLUSH	→
Atomic write of in-memory genome to entity file		
0x36	GHASH	ptr len →
hash_lo hash_hi SipHash-1-3 over genome region; pushes two u64		
0x37	GSIGN	ptr len →
Sign region with entity private key (ed25519)		

ENTITY / IO

0x38	SYSCALL	nr a0..a5 →
ret	Raw OS syscall; args/ret per ABI	
0x39	SPAWN	genome_ptr
genome_len → id	Birth child entity; returns entity ID	
0x3A	SEND	id msg_ptr
msg_len →	Send message to entity by	

ID			
	0x3B	RECV	buf_ptr
buf_len → msg_len		Blocking receive from colony bus	
	0x3C	EMIT	type_tag
data_ptr len →		Emit structured event to event stream	
	0x3D	PCNT	counter_id
→ val		Read hardware performance counter	
	0x3E	SCHED	priority →
		Hint to colony scheduler; non-blocking yield	
	0x3F	DIE	
successor_ptr	successor_len →	Terminate; optionally SPAWN successor first	

5.3 Notable Design Choices

POPCNT at 0x1F. The mutation engine computes Hamming distances between IR node descriptors to bound how "far" a proposed mutation moves.

POPCNT is a single native instruction on x86-64 (BSF/LZCNT family) and ARM64 (CNT). It earns its slot.

ZERO / NZERO at 0x26–0x27. These are not redundant with EQ. ZERO checks the entire 64-bit pattern for all-zero in one instruction path. The fitness tracker calls these thousands of times per second when checking whether a delta improved fitness. They

avoid the cost of pushing a zero immediate before EQ.

GLOG is write-only by design. There is no GREAD for the mutation log. Layer 1 can GHASH the log (to verify integrity) or GSIGN a region of it (for colony trust), but cannot delete or overwrite entries. This is enforced at the Layer 0 evaluation switch, not by policy.

PCNT takes a counter_id. The mapping of counter_id to hardware MSR is platform-specific and handled by Layer 0's syscall layer. Standard IDs: 0 = CPU cycles, 1 = retired instructions, 2 = L1 cache misses, 3 = L2 cache misses, 4 = branch mispredictions, 5 = memory bandwidth. Entities discover available counters at birth and record them in genome metadata.

6. Genome Binary Format

6.1 IR Graph Overview

The genome (Layer 1 and Layer 2) is a typed directed graph. Nodes are operations. Edges are typed data dependencies. Every node carries metadata.

The genome region in the entity file is:

```
[genome_header: 32 B]
[node_table: node_count × 48 B]
[edge_table: edge_count × 16 B]
```

```
[metadata_block: variable]
[string_pool: variable]    ← for
contract/invariant text
```

6.2 Node Descriptor (48 bytes, cache-line friendly)

Offset	Size	Field
0x00	1 B	opcode (u8, from codon table)
0x01	1 B	type_sig (encoded input/output types)
0x02	2 B	in_edge_count (u16)
0x04	2 B	out_edge_count (u16)
0x06	2 B	flags (see below)
0x08	4 B	first_in_edge_idx (u32, index into edge table)
0x0C	4 B	first_out_edge_idx (u32)
0x10	4 B	contract_id (u32, index into string pool; 0 = none)
0x14	4 B	invariant_mask (u32, bitmask of which invariants apply)
0x18	4 B	generation (u32, mutation count for this node)
0x1C	4 B	fitness_score (f32, last measured fitness contribution)
0x20	8 B	perf_samples (u64, packed cycle/miss counts from last eval)
0x28	8 B	mutation_log_offset (u64,

```
byte offset of last GLOG entry)
0x30    0 B    (end – 48 bytes total)
```

Node flags (u16 bitmask):

```
bit 0    deleted (set by GDEL, cleared at
GFLUSH)
bit 1    pinned  (mutation engine must not
touch this node)
bit 2    hot      (set by runtime when call
count exceeds JIT threshold)
bit 3    jit_compiled (Layer 1 JIT has
native version cached)
bit 4    contract_verified (invariant
checker has cleared this node)
bits 5-15 reserved
```

6.3 Edge Descriptor (16 bytes)

Offset	Size	Field
0x00	4 B	src_node_idx (u32)
0x04	4 B	dst_node_idx (u32)
0x08	2 B	src_slot (u16, output slot index on src)
0x0A	2 B	dst_slot (u16, input slot index on dst)
0x0C	2 B	type_tag (u16, data type flowing on this edge)
0x0E	2 B	flags (reserved)

6.4 GMETA Field IDs

When Layer 1 calls `GMETA idx field`, the `field` argument selects which metadata to read:

```
0  generation counter
1  fitness score (as u64 bits of f32)
2  perf_samples packed word
3  mutation_log_offset
4  flags word
5  contract_id
6  invariant_mask
7  in_edge_count
8  out_edge_count
```

Write is `GMETA idx field val →` (three-argument form); reading is `GMETA idx field → val`. Layer 0 distinguishes by stack depth at the opcode boundary.

[OPEN: may split into GMETA_R and GMETA_W]

7. The Ghost Zone

The Ghost Zone is an 8KB page-aligned scratchpad region in the genome, separate from the node and edge tables. It is the only genome region that the mutation engine does not touch — no node descriptors, no edges, no GDEL, no GFLUSH changes to its content structure.

Purpose: emergent persistence. The entity may use this region for anything — internal maps, learned heuristics, cached computations, self-generated notes about past mutations. Its content is opaque to Layer 0. Layer 1 may choose to interpret it however it evolves to interpret it.

The Ghost Zone persists across sessions via the normal GFLUSH mechanism. It is the entity's equivalent of working memory. When first spawned, it is zeroed. What it becomes over the entity's lifetime is entirely the product of evolution.

```
Ghost Zone layout (fixed address, part of
genome mmap):
  Size:      8192 bytes (exactly 2 pages)
  Alignment: 4096-byte page boundary
  Access:    LD / ST / LDB / STB via GADDR
on a reserved ghost_base node
  Mutation:  never touched by the mutation
engine
  GFLUSH:    included in serialization
(contents preserved)
```

8. Memory Model

The entity's genome is memory-mapped via `mmap` at startup. Layer 0 maps the genome region as `PROT_READ | PROT_WRITE`. The kernel machine code

region is mapped `PROT_READ | PROT_EXEC`. These are separate mappings.

Atomic I/O with turn-bit synchronization. When two entities share a genome region (e.g., during horizontal gene transfer via the colony gene pool), concurrent access is coordinated by a turn-bit: a single u64 at a fixed offset at the start of each shared page. The writer sets bit 0 to 1 before writing, clears it after. The reader spins on bit 0. This avoids any need for OS mutex primitives in the hot path.

Cache line discipline. All node descriptors are 48 bytes — not a cache line (64 bytes), but deliberately smaller so that two adjacent nodes fit within three cache lines, minimizing eviction during graph traversal. The edge table entries are 16 bytes — four per cache line.

GFLUSH atomicity. GFLUSH writes the evolved genome back to the entity file using a write-to-temp + atomic rename strategy:

1. Write new genome content to
`<entity_file>.tmp`
2. fsync the tmp file
3. Atomic rename over the original

If the process is killed between steps 2 and 3, the original file is intact. If killed after step 3, the evolved genome is persisted. There is no corruption window.

9. Evolutionary Mechanics

9.1 Bounded Semantic Mutation

The mutation engine (implemented in Layer 1, run by Layer 0) performs three classes of mutations:

Codon swap. Replace a node's opcode with a semantically compatible alternative. Compatibility is defined by the node's type signature and its declared contract. The engine computes the Hamming distance (via POPCNT) between the current opcode and candidate opcodes to bias toward small changes.

Edge rewire. Redirect an edge from one source node to another, preserving type compatibility. This is how the entity discovers new data flow paths — effectively restructuring its algorithm without adding or removing nodes.

Node insertion/deletion. GNEW to allocate a new node, wire it in, evaluate fitness. GDEL to remove a node whose removal improves fitness. Both are constrained by invariant checking after the change.

All mutations are applied speculatively to a copy of the affected subgraph. The invariant checker validates the mutated copy before committing. If validation fails, the copy is discarded. If the fitness improves, the mutation is committed and logged via GLOG.

9.2 Invariants and Contracts

Invariants are boolean conditions declared at birth that must hold after every mutation. They are checked by the invariant checker cell in Layer 1. Example invariants: output type must be [T], cycle count must not exceed N, call depth must not exceed M.

Contracts are per-cell declarations of what a cell must produce given its inputs. The contract is checked by running the mutated cell against a sample of known-correct input/output pairs stored in the genome at birth. If the mutated cell produces different outputs for any sample, the mutation is rejected regardless of fitness improvement.

Contracts are the reason bounded semantic mutation is not the same as classical Linear Genetic Programming. LGP mutates freely and selects by fitness alone. SOMA mutations are additionally constrained to preserve declared behavior.

9.3 Fitness Measurement

Fitness is measured via PCNT hardware counters sampled before and after cell execution. The primary fitness dimension is cycle count per correct invocation. Secondary dimensions (cache misses, branch mispredictions) are recorded in the node's `perf_samples` field.

The entity's genome-level fitness function is declared at birth:

```
fitness {  
    primary:    minimize cycle_count  
    secondary: minimize cache_misses,  
    minimize branch_mispredictions  
    weight:     [1.0, 0.3, 0.2]    ←  
    linear combination  
}
```

The mutation engine tracks a rolling fitness window (last N invocations) per node. Nodes whose fitness has not improved in M mutation attempts are marked for aggressive mutation (edge rewire rather than codon swap). Nodes that consistently improve fitness are pinned (flag bit 1) to protect them from mutation.

9.4 GLOG as Scientific Instrument

Every committed mutation appends a structured record to the GLOG:

```
[timestamp: u64]    ← CPU cycle count at  
commit  
[node_idx: u32]     ← which node was  
mutated  
[mutation_type: u8] ← 0=codon_swap,  
1=edge_rewire, 2=insert, 3=delete  
[old_opcode: u8]    ← before (for  
codon_swap)
```

```
[new_opcode: u8]      ← after  
[fitness_delta: f32] ← fitness change from  
this mutation  
[invariant_result: u8] ← 0=pass,  
nonzero=violation code  
[reserved: 5 B]
```

Total: 24 bytes per entry. A 1GB GLOG holds ~44 million mutation records — sufficient for a long-running entity. The GLOG enables post-hoc analysis of the entity's evolutionary trajectory: fitness curves, mutation strategy distributions, plateau detection, regression events. This is the SOMA contribution to empirical RSI research.

10. Colony Protocol

10.1 Design Decision: Sockets over mmap

Colony communication uses sockets throughout. The alternative — shared mmap with turn-bit synchronization — would constrain entities to the same address space, collapsing the colony to a single machine and a single process. Sockets impose no such constraint. The same `SEND` / `RECV` opcode semantics work whether the target entity is a sibling process, a process on a LAN peer, or a node across a BOINC-style WAN cluster. The transport changes; the opcodes do not.

This also makes the routing strategy evolvable: the `colony_router` entity handles address resolution and is itself a SOMA entity subject to mutation and improvement over its lifetime.

10.2 Entity Addressing

Every entity has two addresses:

entity_id (stable, permanent): A u64 derived from `GHASH` of the entity's initial genome at birth. Globally unique by construction — no coordination or registry required. This ID never changes across the entity's lifetime, even if it moves between machines.

socket_addr (ephemeral): The current `host:port` (TCP) or filesystem path (Unix domain socket) where the entity is listening. Assigned at SPAWN, released at DIE. Stored in the `colony_router`'s routing table, not in the entity's own genome.

Entities address each other exclusively by `entity_id`. The router resolves IDs to socket addresses transparently. An entity has no knowledge of — and no dependency on — where its peers are running.

10.3 Three-Tier Transport

```
Tier 1 – same machine (default)
  Transport:  Unix domain sockets
  Address:
```



```
/tmp/soma/<entity_id_hex_16>.sock
  Latency:      ~1-5  $\mu$ s (kernel-buffered, no
TCP stack)
  Use when:     entities are on the same
host
```

```
Tier 2 – LAN colony
  Transport:    TCP
  Address:      <ip>:<port> (assigned by
OS, registered with router)
  Latency:      ~100-500  $\mu$ s
  Use when:     colony spans multiple
machines on a local network
```

```
Tier 3 – WAN / BOINC-style distributed
colony
  Transport:    TCP between colony_router
peers
  Address:      colony_router on each
machine peers with others
  Latency:      network-dependent
  Use when:     heterogeneous hardware pool;
entities evolve hardware-specific
                variants and share them
across the colony gene pool
```

Layer 0 knows only "send bytes to socket fd" and "recv bytes from socket fd." Tier selection is the router's responsibility, not the kernel's.

10.4 The colony_router Entity

The `colony_router` is a standard SOMA entity – not a kernel primitive – spawned at colony startup. It maintains the routing table in its own genome (Layer 2), which means the routing strategy itself is evolvable. A colony running long enough may develop a smarter router: one that predicts message patterns, pre-establishes connections, or partitions routing by gene pool region.

Responsibilities:

- Accept registration messages from newly SPAWNed entities (`ROUTE_REGISTER`)
- Accept deregistration from dying entities (`ROUTE_DEREGISTER`)
- Forward SEND messages from sender socket to recipient socket
- Bridge Tier 1 → Tier 2 when sender and recipient are on different machines
- Peer with `colony_router` instances on other machines for Tier 3

The router's `entity_id` is declared in the colony genesis genome and known to all entities at SPAWN time – it is the one well-known address in the system.

10.5 SPAWN

```
SPAWN genome_ptr genome_len → entity_id
```

Forks the current entity's process. The child receives a copy of the specified genome region (not the parent's full genome — the child's seed genome is passed explicitly). On startup, the child opens a socket, registers with the `colony_router` (SEND to the well-known router entity_id), and begins listening. The child's entity_id is the GHASH of its initial genome. Parent and child communicate exclusively via SEND/RECV.

10.6 Message Wire Format

TCP is a stream protocol — it does not preserve message boundaries. All messages are prefixed with a 4-byte length field:

Offset	Size	Field
0x00	4 B	msg_len (u32, LE) — total bytes following this field
0x04	8 B	sender_id (u64, LE)
0x0C	8 B	recipient_id (u64, LE)
0x14	4 B	type_tag (u32, LE) — application-defined
0x18	4 B	body_len (u32, LE)
0x1C	N B	body (body_len bytes)

Total header: 28 bytes. `msg_len` = 24 + body_len (excludes the 4-byte length prefix itself).

Unix domain socket messages use the same format for consistency – the framing is identical across all tiers, so the RECV implementation is the same code path regardless of transport.

10.7 Reserved type_tag Values

```
0x00000000  ROUTE_REGISTER    entity →  
router: "I am alive at this socket"  
0x00000001  ROUTE_DEREGISTER entity →  
router: "I am dying"  
0x00000002  ROUTE_QUERY      entity →  
router: "where is entity_id X?"  
0x00000003  ROUTE_REPLY      router →  
entity: response to ROUTE_QUERY  
0x00000010  GENE_OFFER       entity →  
gene pool: "I evolved a useful cell"  
0x00000011  GENE_REQUEST     entity →  
gene pool: "give me cells for contract X"  
0x00000012  GENE_TRANSFER    gene pool →  
entity: cell subgraph bytes + signature  
0x00000013  GENE_ACK         entity →  
gene pool: "accepted / rejected + reason"  
0x00000100  PEER_HELLO       router →  
router: colony peering handshake  
0x00000101  PEER_ROUTE_SYNC  router →  
router: routing table delta sync
```

Values 0x00001000 and above are free for application use.

10.8 Horizontal Gene Transfer

When an entity evolves a cell that significantly improves fitness:

1. `GHASH` the cell's node subgraph → `hash_lo` , `hash_hi`
2. `GSIGN` the subgraph → signature appended to genome
3. `SEND` to the gene pool entity with `type_tag=GENE_OFFER` , body containing subgraph bytes, hash pair, signature, and the `contract_id` the cell satisfies

The gene pool entity (itself a SOMA entity, evolvable) indexes the offer by `contract_id` and hardware profile (derived from the offering entity's PCNT metadata).

When another entity requests cells for a given contract, the gene pool returns the best-performing candidates for that entity's hardware profile.

Receiving entities run their own invariant checker against accepted cells before wiring them in. Cells that fail local invariant checking are rejected and a `GENE_ACK` with the violation code is sent back. The gene pool uses rejection rates to down-rank cells that consistently fail on certain hardware configurations.

The BOINC case: When the colony spans heterogeneous hardware, entities on different machines evolve different Layer 1 variants optimized

for their specific CPUs. The gene pool accumulates hardware-tagged variants of the same logical cell. A new node joining the colony can bootstrap from the gene pool's history — requesting cells that performed well on similar hardware — rather than starting from a naive seed. This compounds over time: the older the colony, the richer the gene pool, the faster new nodes reach maturity.

11. Implementation Roadmap

Each step has exactly one deliverable. Do not move to the next step until the deliverable works.

Step 1 — The bare VM

Deliverable: a Rust binary that loads a hardcoded byte slice, executes it on a u64 stack, and halts.

```
struct SomaVM {
    stack: [u64; 1024],
    sp: usize,           // stack pointer
    pc: usize,           // program counter
    into bytecode
    bytecode: Vec<u8>,
    running: bool,
}
```

Implement the switch loop. Handle only: NOP, HLT, IADD, ISUB, IMUL, JMP, JIF, JNF, CALL, RET. Test with a hardcoded fibonacci bytecode.

Step 2 — Full codon table

Deliverable: all 64 opcodes handled in the switch loop. Genome opcodes (0x30–0x37) panic with "not yet implemented" — that's fine. Write one test per opcode.

Step 3 — Entity file reader

Deliverable: `soma_load(path) -> SomaEntity` that reads the 64-byte header, validates the magic, and maps the Layer 0 / Layer 1 / Layer 2 regions. Use `mmap` via a raw syscall (SYSCALL opcode isn't wired yet — use Rust's `std::fs` for the loader only, since the loader is part of Layer 0 itself, not the VM).

Step 4 — Genome memory map

Deliverable: the genome region is mmap'd as a writable slice. GADDR resolves a node index to a pointer into that slice. LD/ST can read and write node descriptor fields. Write a test that reads opcode field of node 0.

Step 5 — Ghost Zone

Deliverable: the Ghost Zone is mapped at its fixed offset. A test entity writes a u64 into the Ghost Zone,

runs GFLUSH, exits, reloads the entity, reads the Ghost Zone back. The value persists.

Step 6 — PCNT

Deliverable: PCNT opcode calls `rdpmc` (x86-64) or equivalent. `counter_id 0` returns CPU cycles. Wrap in a `#[cfg(target_arch)]` block with a stub for non-x86 targets.

Step 7 — GLOG

Deliverable: GLOG appends a 24-byte record to the log region. GFLUSH serializes the log (write-to-tmp + rename). Verify that log contents survive a restart.

Step 8 — GFLUSH atomic write

Deliverable: GFLUSH does the full write-to-temp + fsync + rename sequence. Kill the process between fsync and rename; verify the original file is intact.

Step 9 — Mutation engine skeleton (Layer 1)

Deliverable: a Layer 1 cell (expressed in SOMA IR, interpreted by Layer 0) that reads node 0's opcode via `GADDR + LD`, increments the generation counter via `GMETA`, and logs the change via GLOG. No actual mutation yet — just the scaffolding.

Step 10 — First codon swap

Deliverable: the mutation engine performs a codon swap on a non-pinned node, evaluates fitness before/after via PCNT, and commits (GFLUSH) or rolls back based on the delta. Write a test entity whose Layer 2 contains an intentionally slow cell. Verify the mutation engine finds and commits a faster equivalent within N invocations.

Step 11a — Unix domain socket SEND/RECV

Deliverable: two entities on the same machine communicate via Unix domain sockets. Entity A SPAWNS entity B. A sends a u64 message (type_tag=0x00000100, body=the u64). B receives it, squares it, sends it back. A prints the result. No router yet — hardcode the socket path.

Step 11b — colony_router + TCP

Deliverable: a `colony_router` entity (minimal — just routing table + forward logic) is spawned first. Two entities register with it on startup. SEND now goes through the router. Swap Unix sockets for TCP loopback (`127.0.0.1:0` , OS-assigned port registered with the router). Verify the square-and-return test still passes through the router.

Step 11c — Cross-machine routing [DEFER until Step 12 is stable]

Deliverable: two `colony_router` instances on different machines peer via `PEER_HELLO`. An entity on machine A sends to an entity on machine B. The routers bridge the message transparently. Test with two VMs or two localhost processes on different port namespaces.

Step 12 — Seed compiler

Deliverable: `soma build seed.soma -> entity.bin`. The seed compiler reads the minimal seed syntax, generates an initial IR graph, packages it with the Layer 0 binary, and writes a valid entity file. The entity file is executable and runs.

12. Workspace Layout

```
kyzu/  
  soma/  
    soma-kernel/      ← Layer 0 Rust  
crate (the VM, no deps)  
  src/  
    main.rs           ← entry point and  
entity loader  
    vm.rs             ← SomaVM struct  
and eval loop  
    codon.rs          ← opcode enum and  
dispatch  
    genome.rs         ← genome mmap,  
node/edge access
```

	glog.rs	← append-only log
	gflush.rs	← atomic write-
back		
	pcnt.rs	← hardware counter
access		
	colony.rs	← SPAWN, SEND,
	RECV, socket layer	
	router.rs	← colony_router
entity bootstrap		
	tests/	
	*.rs	← one test file
per step above		
	soma-seed/	← seed compiler
(parses .soma text, emits entity files)		
	soma-std/	← standard Layer 1
genome (mutation engine, JIT, etc.)		
	soma-router/	← colony_router
genome (evolvable, ships as .soma entity)		
	examples/	
	hello.soma	← simplest
possible seed		
	fibonacci.soma	← canonical test
case		
	colony_two.soma	← two-entity
square-and-return test (Step 11a)		

13. Open Questions

#	Question	Status	Impact
1	GMETA read/write as one opcode or two?	Open	Minor — affects Step 4
2	Colony bus: shared mmap or Unix sockets?	Resolved: sockets	Architecture
3	Sensory cortex (agent use case): fixed Layer 0 memory map or optional Layer 2 genome extension?	Open	Architecture — decide before Step 11a
4	ed25519 for GSIGN: implement from scratch (NIH) or use a well-audited single-file C impl?	Open	NIH purity vs. cryptographic correctness

#	Question	Status	Impact
5	Ghost Zone size: 8KB fixed or genome-declared?	Open	Minor
6	JIT target: x86-64 only first, or abstract codegen from day one?	Open	Scope
7	colony_router discovery: how does a new entity find the router's socket address at startup? Options: (a) well-known Unix socket path, (b) env var, (c) hardcoded in the entity's	Open	Required before Step 11b

#	Question	Status	Impact
	genome at SPAWN time		
8	RECV timeout: blocking indefinitely, or genome-declared timeout with a fallback opcode path?	Open	Minor — affects Step 11a
9	Gene pool: dedicated entity or a role any entity can advertise?	Open	Decide before Step 11b
10	WAN encryption: plaintext TCP for local/trusted colonies, TLS for cross-machine? TLS conflicts with NIH if it	Open	Defer until Step 11c

#	Question	Status	Impact
	requires a crate		

SOMA spec v0.3 – colony protocol updated to socket-based architecture with three-tier transport and evolvable colony_router. Supersedes v0.2.